GPU 内存层次结构

GPU Memory Hierarchy

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Shared Memory	共享内存	
Coalescing	合并访问	
Registers	寄存器	
Tiling	分块	
GPU Memory Hierarchy	GPU 内存层次结构	
Data Movement	数据移动	
Memory Hierarchy	内存层次结构	
Performance	性能	
Core	核心	
Global Memory	全局内存	
Register	寄存器	
Memory Access	内存访问	
Device Memory	设备内存	
L2 Cache	二级缓存	
SM	流式多处理器	SM
Tensor	张量	
Synchronization	同步	
Throughput	吞吐量	
Latency	延迟	
Kernel	内核	
Arithmetic Intensity	算术强度	
Mental Model	理解框架	
Occupancy	占用率	
Profiling	性能剖析	
Local Memory	局部内存	
Attention	注意力机制	Attention
Warp	线程束	

英文原词	中文译名	缩写／符号
GEMM	通用矩阵乘法	GEMM
Generation	生成	
FlashAttention	FlashAttention（方法名保留）	
Vocabulary	词表	
Inference	推理	
Training	训练	
Prefill	预填充	
Decode	解码	
Values	值	
Models	模型	
Activation	激活	
Model	模型	
Texture Cache	纹理缓存	
Register Pressure	寄存器压力	
Matrix	矩阵	
Dot Product	点积	
Residency	驻留状态	
Spilling	溢出	
Register File	寄存器文件	
Resident	驻留	
Warps	线程束	
Memory Traffic	内存流量	
Coalesced Access	合并访问	
GPU Generation	GPU 代际	
Eviction	逐出	
Thread Block	线程块	
Working Set	工作集	
Memory Bandwidth	内存带宽	
Memory Footprint	内存占用	

英文原词	中文译名	缩写／符号
Asynchronous Copy	异步复制	
Operators	算子	
Routing	路由	
Cache Hit	缓存命中	
Profiler	性能剖析器	
Cache Line	缓存行	
Cache Capacity	缓存容量	
Normalization	归一化	
Tokens	词元	
Tensor Shape	张量形状	
Wall-clock Time	实际耗时	
Floating-point	浮点	
FLOPs	浮点运算次数	
Tensor Cores	张量核心	
Matrix Multiplication	矩阵乘法	
FLOP Count	浮点运算次数	
Vector	向量	
FP32	32 位浮点	FP32
FLOP	浮点运算	
Arithmetic Work	算术工作量	
Bandwidth-bound	带宽受限	
Launch Overhead	启动开销	
Compute-bound	计算受限	
Precision	精度	
Roofline Model	屋顶线模型	
Scaling	缩放	
Dense Attention	稠密注意力	
Softmax	Softmax 归一化	Softmax
Score Matrix	分数矩阵	

英文原词	中文译名	缩写／符号
Combine	合并	
Recomputation	重计算	
Operator	算子	
Head Dimension	头维度	
Memory-bound	内存受限	
Query	查询	Q
KV Cache	键值缓存	KV Cache
Token	词元	
Keys	键	
Forward Pass	前向传播	
Gradient	梯度	
Batch Size	批大小	
Continuous Batching	连续批处理	
Offloading	卸载	
Fused Kernel	融合内核	
Tensor Core	张量核心	
High-bandwidth Memory	高带宽内存	HBM
Pipeline	流水线	
Wavefront	波前	
GPU Generations	GPU 代际	
Serving	推理服务	
Index	索引	
Exact Attention	精确注意力	

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

GPU 内存层次结构 · 第 8 页

如何阅读本手册 · 第 9 页

内容 · 第 10 页

内容续 · 第 11 页

1 为什么 GPU 需要内存层次结构（Memory Hierarchy） · 第 12 页

2 层次结构一览 · 第 13 页

3 容量、流量、带宽和延迟（Latency） · 第 15 页

4 寄存器（Registers）：最接近的工作状态 · 第 16 页

5 寄存器（Register）文件是有限的资源 · 第 17 页

6 本地内存和寄存器（Register）溢出 · 第 18 页

7 共享内存（Shared Memory）：显式片上存储 · 第 19 页

8 共享内存（Shared Memory）不仅仅是另一个缓存 · 第 20 页

9 L1：小型、依赖于架构的第一级缓存 · 第 21 页

10 L2：HBM 之前的设备范围内重用 · 第 22 页

11 HBM 和设备内存（Device Memory） · 第 23 页

12 HBM 不是全局内存（Global Memory）的同义词 · 第 24 页

13 负载在层次结构中的旅程 · 第 25 页

14 时间局部性 · 第 26 页

15 空间局部性 · 第 27 页

16 工作组和复用距离 · 第 28 页

17 缓存命中、未命中和下游流量 · 第 29 页

18 合并：组织线程束（Warp）的内存请求 · 第 30 页

19 工作示例：连续负载和跨步负载 · 第 31 页

20 平铺将容量转化为再利用 · 第 32 页

21 GEMM 也是一个内存重用问题 · 第 33 页

移动 22 字节与执行算术运算 · 第 34 页

23 为什么元素内核和 GEMM 的行为不同 · 第 35 页

24 注意力机制（Attention）也是一个数据移动问题 · 第 36 页

25 FlashAttention 作为层次结构感知示例 · 第 37 页

26 以不同的方式预填充（Prefill）和解码（Decode）压力记忆 · 第 38 页

27 权重、激活和 KV 状态具有不同的生命周期 · 第 39 页

28 训练（Training）和推理（Inference）移动不同的字节 · 第 40 页

29 容量和带宽是独立的硬件属性 · 第 41 页

30 理论带宽与实际带宽 · 第 42 页

31 缓存友好并不自动意味着速度快 · 第 43 页
32 高带宽并不意味着低延迟（Latency） · 第 44 页
33 NVIDIA 和 AMD：相同的问题，不同的词汇 · 第 45 页
34 代架构改变了细节 · 第 46 页
35 种常见误解 · 第 47 页
36 一套实用的内存理解框架（Mental Model） · 第 48 页
参考文献和进一步阅读 · 第 49 页
参考图：针对哪个问题阅读什么内容 · 第 52 页

GPU 内存层次结构

对应原稿第 1 页

了解 GPU 内存层次结构（GPU Memory Hierarchy）

从寄存器（Registers）和共享内存（Shared Memory）到 L2、HBM 和数据移动（Data Movement）

> 关于 AI 工作负载在何处保存数据以及这些字节如何移动的技术手册：寄存器（Registers）、共享内存（Shared Memory）、L1/L2 缓存、HBM、局部性、工作集、合并、平铺以及现代 GPU 内核背后的内存行为。

如何阅读本手册

对应原稿第 2 页

当四个问题分开时，GPU 内存最容易推理：

1、什么是逻辑内存空间？ CUDA 命名空间，例如寄存器（Register）、本地、共享和全局内存（Global Memory）。这些名称描述了可见性、生命周期和编程语义，而不是简单的物理阶梯。[1]

2. 数据从哪里获取？全局负载可以通过 SM 侧 L1/TEX 路径、GPU 范围的 L2 缓存或设备内存（Device Memory）来满足。本地内存访问（Memory Access）在范围内是线程私有的，但仍然使用设备内存支持。[1][3]

3.实际移动了多少字节？张量（Tensor）大小就是容量。流量是通过特定接口传输的数量。当重复使用或重复获取数据时，两者可能存在数量级的差异。

4. 是什么限制了进展？容量、延迟（Latency）、带宽、请求吞吐量（Throughput）、指令发布、同步（Synchronization）和计算吞吐量是不同的约束。内核（Kernel）可以轻松地适应 HBM，并且仍然花费大部分时间等待内存移动。

> 核心区别。内存层次结构（Memory Hierarchy）不仅仅是从“快”到“慢”的列表。性能（Performance）取决于移动哪些字节、移动的频率、哪个级别提供它们以及是否存在足够的独立工作来重叠等待。

范围。本手册建立了理解人工智能系统其余部分所需的记忆理解框架（Mental Model）。算术强度（Arithmetic Intensity）、Roofline 分析、详细的合并规则、共享内存库冲突、寄存器（Register）/占用调整、内核（Kernel）融合和分析稍后都会收到专门的手册。在这里，它们出现的距离仅足以使记忆行为变得连贯。

供应商大会。 NVIDIA CUDA 术语用于具体示例，因为它在人工智能系统中很常见。 AMD CDNA 术语用作交叉检查，因此一般原则不会以 CUDA 法律的形式呈现。[12][16]

单位。当计算明确使用 2 的二进制幂时，本手册使用 KiB、MiB 和 GiB。产品规格使用供应商打印的单位（例如 GB 和 TB/s）复制，而不是默默地转换标签。 [2]

内容

对应原稿第 3 页

1 为什么 GPU 需要内存层次结构（Memory Hierarchy） 5 2 层次结构概览 6 3 容量、流量、带宽和延迟（Latency） 7 4 寄存器（Registers）：最接近的工作状态 8 5 寄存器文件是有限资源 9 6 本地内存和寄存器溢出 10 7 共享内存（Shared Memory）：显式片上存储 11 8 共享内存不仅仅是另一个缓存 12 9 L1：一个小型的、依赖于架构的第一缓存 13 10 L2：HBM 之前的设备范围重用 14 11 HBM 和设备内存（Device Memory） 15 12 HBM 不是全局内存（Global Memory）的同义词 16 13 负载在层次结构中的旅程 17 14 时间局部性 18 15 空间局部性 19 16 工作集和重用距离 20 17 缓存命中、未命中和缓存命中下游流量 21 18 合并：组织 warp 的内存请求 22 19 工作示例：连续和跨步加载 23 20 平铺将容量转化为重用 24 21 GEMM 也是一个内存重用问题 25 22 移动的字节与执行的算术 26 23 为什么元素内核和 GEMM 的行为不同 27 24 注意力机制（Attention）也是一个数据移动问题 28

内容续

对应原稿第 4 页

25 FlashAttention 作为层次结构感知示例 29 26 以不同的方式预填充（Prefill）和解码（Decode）压力内存 30 27 权重、激活和 KV 状态具有不同的生命周期 31 28 训练（Training）和推理（Inference）移动不同的字节 32 29 容量和带宽是独立的硬件属性 33 30 理论带宽与实现的带宽 34 31 缓存友好并不自动意味着快速 35 32 高带宽并不意味着快速平均低延迟（Latency） 36 33 NVIDIA 和 AMD：相同的问题，不同的词汇 37 34 各代架构改变了细节 38 35 常见误解 39 36 实用的记忆理解框架（Mental Model） 40 参考资料和进一步阅读 41 参考图：针对哪个问题阅读什么内容 42

> 第 1-32 章重点讨论持久概念。第 33 章翻译了不同供应商的术语。第 34 章特意针对特定代际，在硬件文档更改时应重新检查。

1 为什么 GPU 需要内存层次结构（Memory Hierarchy）

对应原稿第 5 页

GPU 可以执行大量算术运算，但算术单元只能处理已到达它们的数据。因此，存储系统必须立即解决相互冲突的目标：为大型模型（Models）保留足够的总字节，以高聚合速率交付这些字节，并使经常重用的值靠近执行单元可用。

没有任何一种内存技术能够同时满足这三点。非常大的内存池在物理上距离算术管道较远，并且在与小型片上存储相同的访问特性下提供的成本昂贵。管道附近的非常小的结构可能非常有用，但它们不能容纳整个模型（Model），甚至不能容纳大的激活（Activation）张量（Tensor）。

因此，现代 GPU 使用存储层。线程私有值通常存在于寄存器（Registers）中。块中的线程可以通过共享内存（Shared Memory）显式地进行协作。硬件管理的缓存无需显式分段即可捕获一些重用。设备内存（Device Memory）（通常是数据中心 AI 加速器上的 HBM）提供更大的容量和非常高的聚合带宽。[1][5][6]

重要的词是重用。如果从设备内存（Device Memory）中获取一个值并使用一次，则层次结构几乎没有机会分摊该移动。如果相同的值可以在芯片上使用数十或数百次，则相对昂贵的传输可以提供大量计算。

这就是为什么具有相同数学运算次数的两个内核可以以非常不同的速度运行。一种映射可以连续地从设备存储器获取数据。另一种可能会重新组织工作，以便每个获取的图块在被替换之前参与许多操作。

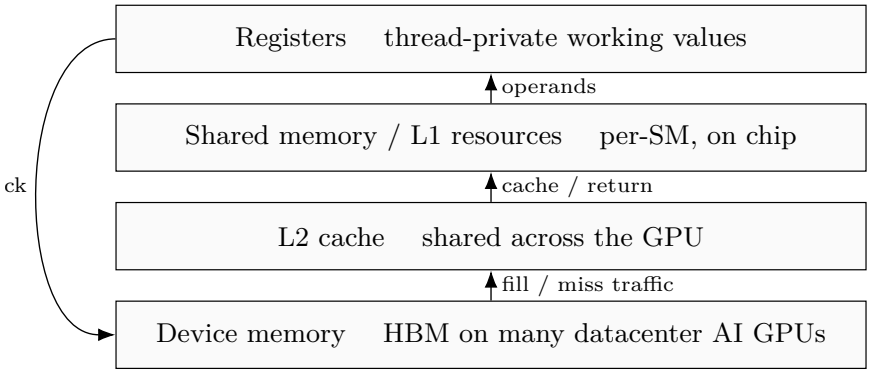
> 理解框架（Mental Model）。 GPU 内存性能（Performance）在很大程度上是将有用的重用放置在层次结构的正确级别，同时避免级别之间不必要的流量的问题。

本手册的其余部分从最接近线程的存储向外构建该模型（Model）到设备内存（Device Memory）。

2 层次结构一览

对应原稿第 6 页

下图是有意示意性的。它显示了 AI 数据的通用路径，而不是保证每个 GPU 上的每条指令都完全遵循这些框。



设备内存（Device Memory）

共享内存（Shared Memory）/L1资源

寄存器（Registers）

二级缓存（L2 Cache）

许多数据中心 AI GPU 上的 HBM

线程私有工作值

跨 GPU 共享

操作数

缓存/返回

填补/错过交通

per-SM，片上

存储/写回

三个警告使这张图片保持诚实。

首先，编程空间和物理结构并不相同。 CUDA 全局内存（Global Memory）是整个网格可见的地址空间。全局负载可以命中片上缓存。 CUDA 本地内存是一个线程私有的，但其后备存储位于设备内存（Device Memory）中。[1][3]

其次，层次结构依赖于体系结构。在最新的 NVIDIA 数据中心 GPU 上，L1/纹理缓存（Texture Cache）和共享内存（Shared Memory）来自统一的片上资源，其分区可以调整。 AMD CDNA 公开了具有每 CU L1 缓存和本地数据共享 (LDS) 的不同组织。[5][6][12]

第三，更接近并不意味着自动更好。将数据移入共享内存（Shared Memory）需要指令并且通常需要同步（Synchronization）。在寄存器（Registers）中保留更多值会增加寄存器压力（Register Pressure）。缓存可能包含从未重用的数据。仅当访问模式值得其成本时，每个级别才有帮助。

将层次结构视为一组可以保留和重用数据的位置，而不是每个级别有一个固定延迟（Latency）数的通用排名。

3 容量、流量、带宽和延迟（Latency）

对应原稿第 7 页

这四个量常常被归结为模糊的词“记忆”。他们回答不同的问题。

容量是一个存储级别可以容纳多少字节。 BF16 中形状为$[4, 4096, 8192]$的张量（Tensor）包含

$$4 \times 4096 \times 8192 \times 2 = 268{,}435{,}456 \text{ bytes} = 256 \text{ MiB}.$$

这是一个存储要求，而不是关于内核（Kernel）运行时将移动多少数据的声明。

流量是通过特定接口传输的字节数。如果读取不由缓存或其他重用机制提供服务，则 256 MiB 张量（Tensor）读取十次可以创建远远超过 256 MiB 的下游流量。

带宽是一个速率：

带宽 = 传输的字节数

.

时间

峰值带宽是特定条件下的硬件上限。获得的带宽取决于访问模式、请求并发性、缓存行为、争用和内核（Kernel）本身。[2][3]

延迟（Latency）是启动访问和将其结果提供给相关工作之间的延迟。系统可以具有高带宽，而个人访问仍然具有相当大的延迟。 GPU 通过保持许多独立操作的运行并在某些请求等待时安排其他工作来部分容忍这种情况。

数量	提问它回答
Capacity	How much data can reside here?
Traffic	How many bytes crossed this interface?
Bandwidth	How quickly can bytes cross over time?
Latency	How long until a particular dependency can be satisfied?

容量 流量 带宽 延迟（Latency）

这里可以驻留多少数据？有多少字节穿过该接口？字节随时间变化的速度有多快？需要多长时间才能满足特定的依赖关系？

> 工作负载可以适合 HBM 并且仍然受到带宽限制。它可以实现高带宽，但在暴露的独立工作太少时仍然会受到延迟（Latency）的影响。容量、流量、带宽和延迟不应用作同义词。

4 寄存器（Registers）：最接近的工作状态

对应原稿第 8 页

寄存器（Registers）保存线程执行指令直接所需的值：循环计数器、指针、矩阵（Matrix）块片段、中间算术结果和累加器。 CUDA 自动标量变量通常驻留在寄存器中，除非编译器选择其他位置。[1]

寄存器（Registers）不是线程共同浏览的可寻址缓存。它们属于各个线程的执行状态。一个线程不能简单地通过普通内存地址加载另一个线程的寄存器值。因此，线程之间的通信使用其他机制，例如共享内存（Shared Memory）、warp 级操作或特定于体系结构的集体指令。

这使得寄存器（Registers）成为非常细粒度的重用的理想选择。考虑一个累积部分点积（Dot Product）的线程：

$$a \leftarrow a + x_i y_i.$$

将 a保存在寄存器（Register）中意味着每个乘法累加可以更新相同的附近状态，而无需在每次迭代后将部分和写入设备内存（Device Memory）。

矩阵（Matrix）内核将这一点推得更远。 CUTLASS 描述了 warpand 线程级 GEMM 结构，其中操作数从共享内存（Shared Memory）加载到寄存器（Registers）支持的片段中，并且累加器保留在许多矩阵指令的寄存器中。 [14]

好处并不是“寄存器（Registers）是免费的”。它们是有限的硬件资源，并且参与调度约束。但是，当某个值在一个线程内具有较高的重用性时，将其保持在寄存器状态可以避免通过较大内存结构的重复流量。

> 安全措辞。寄存器（Registers）是接近执行的线程私有工作值的正常所在地。避免通用的说法，例如“寄存器访问始终是一个周期”。确切的时序取决于指令路径、依赖性、存储、调度和架构。

本手册将寄存器（Registers）视为内存层次结构级别，因为它们确定临时状态是保持接近计算还是溢出到可寻址内存系统中。占用后果在 AS09 中得到专门处理。

5 寄存器（Register）文件是有限的资源

对应原稿第 9 页

寄存器文件（Register File）总体上可能很大，但仍然成为一个约束，因为数千个常驻线程共享它。NVIDIA 当前的计算能力表列出了几种最新架构的每个 SM 64K 32 位寄存器（Registers），而 Hopper H100 允许每个线程最多 255 个寄存器。[4][5]

假设内核（Kernel）每个线程使用 128 个寄存器（Registers）并启动 256 个线程块。暂时忽略分配粒度和其他架构限制，一个块将请求

$$128 \times 256 = 32{,}768$$

32 位寄存器（Registers）。 64K 寄存器 SM 无法同时托管任意数量的此类块。因此，即使共享内存使用量很小，寄存器需求也会影响可驻留（Resident）的工作量。

这造成了真正的权衡。每个线程更多的寄存器（Registers）可以使有用的临时变量和累加器靠近算术管道。但较大的寄存器占用空间可以减少驻留（Resident）线程束（Warps）的数量，从而减少可用于隐藏停顿的独立工作池。

相反的策略也可能适得其反。强制编译器使用更少的寄存器（Registers）可能会增加占用率（Occupancy），但会产生额外的本地内存流量（Memory Traffic），因为值不再适合寄存器状态。在这种情况下，内核（Kernel）可能会获得更多常驻线程束（Warps），但每个线程束会执行更多内存工作。

> 寄存器压力（Register Pressure）本身并不是坏事。这是一种资源权衡。相关问题是寄存器（Register）占用空间是否能够实现有用的重用和指令级并行性，而不会产生足够的溢出（Spilling）或驻留损失来损害内核（Kernel）。

这就是为什么“使用更少的寄存器（Registers）”和“最大化占用”不是通用的优化规则。 AS09 将直接处理该交互。这里的内存教训更简单：有限的寄存器预算决定了在层次结构的最接近的级别可以保留多少线程私有状态。

6 本地内存和寄存器（Register）溢出

对应原稿第 10 页

CUDA 的术语“本地内存”很容易被误读。“本地”描述了范围：每个线程都有自己的本地地址空间。这并不意味着存储在物理上靠近该线程的执行单元。当前的 CUDA 文档将本地内存放置在设备内存（Device Memory）中。[1]

编译器可以将本地内存用于具有动态索引的数组、大型线程私有对象或无法保留在寄存器（Registers）中的值。最后一种情况通常称为寄存器溢出（Spilling）。[1][3]

从概念上讲：

不保留在寄存器（Registers）中

线程临时

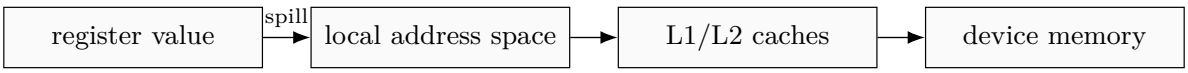
$$\xrightarrow{\text{does not remain in registers}}$$

本地地址空间.

当数据存在于本地存储器时，可以通过缓存提供服务。然而，如果未命中，请求将通过层次结构进一步传播到设备内存（Device Memory）。这就是为什么溢出并不等于“每次一次 HBM 访问”，但它仍然会增加负载、存储、缓存压力和长延迟（Latency）依赖性

当值保留在寄存器（Register）中时，该值不存在。

线程私有范围并不意味着分散的物理请求。 CUDA 跨通道布置本地存储，以便线程访问相同的每线程偏移量的线程束（Warp）仍然可以生成合并访问（Coalesced Access）。例如，即使每个通道拥有不同的逻辑对象，每个通道读取其自己的溢出数组的相同槽都可以得到有效的服务。[1]



寄存器（Register）值

泄漏

本地地址空间

L1/L2 缓存

设备内存（Device Memory）

> 不要仅仅因为它是线程本地的，就在物理层次结构中的寄存器（Registers）旁边绘制“本地内存”。它的逻辑范围是本地的。它的后备存储位于设备内存系统中。

7 共享内存（Shared Memory）：显式片上存储

对应原稿第 11 页

共享内存（Shared Memory）为块中的线程提供了一个显式管理的位置来交换和重用数据。 CUDA 通过块作用域公开它，并将存储放置在 SM 上。在最新的 NVIDIA GPU 上，它是从 L1/纹理缓存使用的相同统一片上资源中提取的。[1][5][6]

这种显式控制就是定义属性。硬件缓存自动决定保留哪些行。共享内存（Shared Memory）根据内核（Kernel）的程序逻辑进行填充、索引、同步和覆盖。

一个常见的模式是：

1. 从设备内存（Device Memory）中协同加载图块，

2. 同步以便图块准备就绪，

3. 在许多操作中重复使用图块，

4.工作完成后，将其更换为下一块瓷砖。

回报来自重用。如果 256 个线程中的每个线程都需要许多相邻线程也需要的值，则获取一个共享图块可以替代大量冗余的下游负载。

共享内存（Shared Memory）也有成本。它消耗有限的 per-SM 容量，可以减少块驻留，在线程通信时需要正确的同步（Synchronization），并且具有可以序列化某些模式的存储访问行为。这些存储体冲突详细信息属于 AS08。

Hopper H100 和 Blackwell B200 均支持 256 KB 统一 L1/纹理/共享资源内的可配置共享内存分割，每个 SM 具有高达 228 KB 的共享内存容量，并且由这些特定设备上的一个选择加入的块可寻址高达 227 KB。[5][6]

> 总体原则，具体实施。“共享内存（Shared Memory）是程序员管理的片上存储”是持久的。确切的容量、剥离选择、存储体行为和集群级功能取决于 GPU 一代。

8 共享内存（Shared Memory）不仅仅是另一个缓存

对应原稿第 12 页

共享内存（Shared Memory）和缓存都可以使数据接近计算，但它们解决问题的方式不同。

	共享内存	硬件缓存
定位终生沟通	Kernel chooses what to stage Explicit shared-memory address Managed by kernel/block logic Can exchange data among cooperating threads	Hardware reacts to accesses Ordinary address space Managed by replacement policy Primarily serves memory accesses
主要风险	Synchronization, bank conflicts, capacity pressure	错过、驱逐、有限区域

内核（Kernel）选择要暂存的内容 显式共享内存地址 由内核/块逻辑管理 可以在协作线程之间交换数据 同步（Synchronization）、存储体冲突、容量压力

硬件对访问作出反应 普通地址空间 由替换策略管理 主要服务于内存访问

将共享内存（Shared Memory）称为“缓存”有时是有用的类比，因为程序员经常使用它来保留否则会重复获取的数据。但如果这种类比隐藏了软件责任，就会产生误导。

例如，矩阵（Matrix）内核（Kernel）可以确切地知道线程块（Thread Block）将使用哪个 A和 B图块。将这些图块暂存在共享内存（Shared Memory）中可实现确定性重用。通用硬件缓存不接收该高级意图。它看到内存请求流并应用自己的策略。

另一方面，明确的分期并不总是值得做的。如果数据几乎没有重用，如果访问已经在缓存中命中，或者如果暂存添加了太多同步（Synchronization）和复制，则共享内存版本可能会比更简单的缓存路径慢。

> 正确的区别是控制。共享内存（Shared Memory）公开了软件控制的重用机制。缓存公开了硬件控制的重用机制。它们可能在某些架构上共享物理资源，但它们的编程语义仍然不同。

这种区别在 FlashAttention、GEMM、转置、归约和许多融合内核（Kernel）中变得很重要。内核编写者正在决定哪些中间体值得在片上进行可预测的驻留，而不是希望缓存替换策略将它们保留下来。

9 L1：小型、依赖于架构的第一级缓存

对应原稿第 13 页

L1 是 SM 附近许多普通内存路径遇到的第一个硬件管理缓存。它的确切组织随着体系结构的不同而变化，不同的内存空间可以以不同的方式与之交互。因此，Nsight Compute 将 L1/TEX 作为一个架构单元，而不是假装存在一种通用的教科书缓存设计。 [3]

在最新的 NVIDIA 数据中心架构上，L1/纹理缓存（Texture Cache）和共享内存（Shared Memory）容量是统一片上资源的一部分。 Hopper H100 和 Blackwell B200 具有相同的 256 KB 最大组合容量，而为共享内存保留的数量可以在支持的剥离选择中根据内核（Kernel）进行更改。[5][6]

有用的 L1 命中可以减少 L2 和设备内存（Device Memory）的流量。这很重要，因为下游接口由更多请求者共享，并且距离发出 SM 更远。 Nsight 的内存分析准确地报告请求的扇区、L2 未命中、命中率和链接利用率，因为高级“负载计数”并不能揭示有多少流量逃逸第一个缓存。 [3]

但L1命中率并不是通用的性能（Performance）分数。内核（Kernel）可以具有很高的命中率，但仍然受到指令问题、序列化、共享内存冲突或某些其他资源的限制。相反，如果流内核的访问被合并并且设备带宽得到有效利用，则流内核可能有意地具有很少的时间重用并且仍然表现良好。

> 架构警告。不要将一种 L1 大小或一种缓存规则记忆为“GPU L1”。持久的想法是，小型片上硬件管理结构可以在消耗 L2 或 HBM 流量之前满足某些请求。必须检查目标架构的确切容量和行为。

10 L2：HBM 之前的设备范围内重用

对应原稿第 14 页

L2 位于层次结构的较深处，并在 GPU 之间共享，而不是属于一个 SM。 Nsight 将其描述为片上内存客户端和设备内存（Device Memory）之间的缓存，并报告继续发送到设备、系统或对等内存的 L2 请求、扇区、命中率、吞吐量（Throughput）和未命中率。 [3]

此位置使得 L2 对于跨块或 SM 边界的重用非常有用。当另一个块需要相同区域时，从 SM 的 L1 驱逐的图块可能仍然在 L2 中可用。因此，即使软件没有明确协调重用，重复的模型（Model）权重读取、共享查找表和重叠张量（Tensor）图块也可以受益。

L2 仍然是有限的。如果工作集（Working Set）比缓存大得多，或者如果许多不相关的流竞争它，则有用的行可能会在重用之前被逐出。例如，64 MiB 的热区域不能完全驻留在 H100 标称的 50 MB L2 内。这并不意味着缓存命中为零。它只是意味着整个区域不能同时保留。[5]

从 Ampere 级 CUDA 器件开始，软件还可以为 L2 中选定的全局内存区域提供持久性提示。这些机制影响替换优先级，而不是将 L2 转换为显式寻址的暂存存储器。[1]

> L2可以在不改变逻辑张量（Tensor）大小的情况下减少HBM流量。这就是为什么“张量是 4 GB”几乎没有告诉您在内核（Kernel）期间实际穿过 HBM 接口的字节数。

下一个层次是设备存储器本身，其中大容量和巨大的聚合带宽共存，并且移动成本比片上重用高得多。

11 HBM 和设备内存（Device Memory）

对应原稿第 15 页

高带宽内存（或 HBM）是许多数据中心加速器使用的物理内存技术。它旨在提供非常宽的接口和接近 GPU 封装的高聚合带宽。例如，H100 SXM 指定为 80 GB HBM3 和 3.35 TB/s 内存带宽（Memory Bandwidth）。 H200 SXM 将 HBM3e 的容量提升至 141 GB，速度为 4.8 TB/s。[7][8]

与普通的 CPU 内存通道相比，这些数字非常巨大，但从 SM 的角度来看，HBM 仍然是设备上层次结构的远端。即使算术单元能够执行更多工作，在 HBM 和计算之间重复移动相同值的内核（Kernel）也会受到该流量的限制。

HBM容量和HBM带宽是独立的规格。更大的容量可以让更大的模型（Model）或工作集（Working Set）保留在设备上。更大的带宽可以让内存系统维持更高的传输速率。增加一个在数学上并不意味着另一个。

HBM 也不是通用的。某些 GPU 使用 GDDR 而不是 HBM。保持稳定的编程概念是设备内存（Device Memory）：支持全局分配和其他地址空间的大型片外内存。 HBM 是当前许多人工智能加速器中该角色的一种实现。

> HBM 中的模型（Model）拟合回答了容量问题。它没有回答权重、激活或 KV 状态将被读取多少次，访问是否会命中缓存，或者是否可以在延迟（Latency）目标内维持所需的流量。

这种区别在推理（Inference）中尤其重要，其中模型（Model）可以轻松地适合内存，但在解码（Decode）期间仍然受到重复流权重或 KV 状态的限制。

12 HBM 不是全局内存（Global Memory）的同义词

对应原稿第 16 页

CUDA 的全局内存（Global Memory）是跨线程可见的编程模型（Model）地址空间。 HBM是一种物理内存技术。将它们混为一谈会产生几个错误的结论。

来自全局内存（Global Memory）的加载不一定达到 HBM。如果所请求的数据已经存在于合适的缓存中，则可以在层次结构中较早地满足该请求。相反，来自线程私有本地地址空间的数据最终可以由用于全局分配的相同设备内存（Device Memory）支持。[1][3]

其区别可写为：

地址空间语义= 此特定访问的物理服务位置.

这在解释分析器时很重要。内核（Kernel）可以执行许多全局加载指令，同时生成更少的 HBM 流量，因为 L1 或 L2 提供重复的请求。相反的情况也可能发生：合并不佳或重用率低的访问可能会产生比源代码隐含的有用字节多得多的下游流量。

在讨论“内存使用”时它也很重要。模型（Model）分配的全局内存占用（Memory Footprint）是一种容量度量。缓存占用是一种暂时的硬件状态。寄存器（Register）和共享内存的使用是每个内核（Kernel）的资源需求。它们不应该像一个平池一样随意地加在一起。

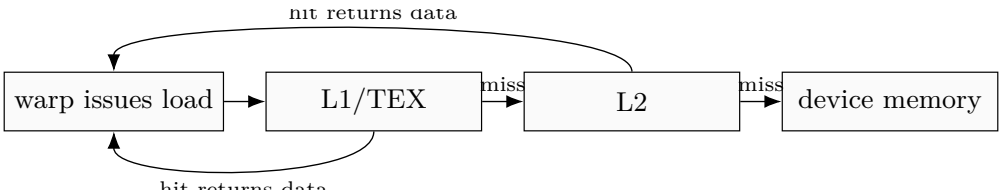
> 实用语言。在讨论 CUDA 地址空间时说“全局内存加载”。在讨论字节的物理服务或传输位置时，请说“L1 命中”、“L2 命中”或“HBM/设备内存流量（Memory Traffic）”。

本手册的其余部分一致地使用了这种区别。它让我们讨论单个张量（Tensor），同时单独推理其逻辑位置、缓存行为和通过硬件的实际移动。

13 负载在层次结构中的旅程

对应原稿第 17 页

对于最近的 NVIDIA GPU 上的普通可缓存全局加载，一个有用的教学路径是：



线程束（Warp）问题负载

命中返回数据

L1/特克斯

想念

想念

设备内存（Device Memory）

命中返回数据

这是一个理解框架（Mental Model），而不是通用的路由（Routing）图。缓存运算符、纹理/常量路径、异步复制（Asynchronous Copy）机制、对等内存和特定于体系结构的功能可以更改路由。[1][3]

重要的一点是失误意味着什么。如果 L1 无法提供所请求的扇区，则该请求会消耗下游的容量和带宽。如果 L2 也未命中，设备内存（Device Memory）必须提供数据。因此，缓存命中（Cache Hit）可以节省下游流量，并且通常可以避免与更远的传输相关的一些延迟。

多个线程束（Warps）和多个请求可以同时进行。当依赖项等待时，调度程序可以发出不相关的工作。这就是 GPU 能够容忍大量访问延迟（Latency）而不减少请求本身的物理延迟的方式。

因此，分析器需要的不仅仅是“负载计数”。 Nsight 跨 L1、L2 和 DRAM 接口公开请求、扇区、命中率、字节和利用率。这些计数器有助于回答流量在哪里扩展、在哪里捕获重用以及哪个链路接近其吞吐量（Throughput）上限。 [3]

> 当内核（Kernel）速度缓慢时，请提出两个单独的问题：重要的请求传输了多远？是否有足够的独立工作来重叠等待？

14 时间局部性

对应原稿第 18 页

时间局部性意味着数据在最近被访问后会被重用，因此将其保留在附近可能会有回报。

考虑在短时间内由多个线程块使用的权重图块。第一个请求可以从设备存储器中获取它。如果该图块保留在 L2 中，则后面的块可以重用它，而无需再次读取 HBM。张量（Tensor）本身没有改变。只有可重用副本的位置有。

同样的原理也出现在较小的尺度上。加载到寄存器（Register）中的值可以参与许多算术指令。共享内存块可以供给许多线程。缓存行（Cache Line）在驱逐之前可以满足多次访问。

仅当重用在存储级别需要空间用于其他内容之前到达时，时间局部性才有用。这就是缓存容量（Cache Capacity）和周围访问流很重要的原因。在接触大量不相关的数据后重用一个值对于小型缓存来说可能已经来不及保留它了。

推理时间重用的一个简单方法是问：

- 多久之后再次需要该值？
- 在此之前有多少其他数据竞争同一存储？
- 重用对于硬件缓存策略是否可见，或者软件是否应该显式地暂存该值？

> 时间局部性并不是“同一个地址出现两次”。它是在时间尺度和工作集（Working Set）上发生的重用，其中层次结构的某些级别可以实际保留数据。

AI 工作负载包含多个尺度的时间局部性：跨词元（Tokens）和批处理元素的重复权重、内核（Kernel）内重用的矩阵（Matrix）图块、跨位置共享的标准化参数以及跨解码步骤的 KV 状态读取。哪个级别捕获该局部性取决于重用的大小和时间。

15 空间局部性

对应原稿第 19 页

空间局部性意味着附近的地址被足够紧密地一起访问，使得以块的形式获取数据是有用的。

对于每个源级加载，缓存和内存接口通常不会从 HBM 移动一个任意标量。它们传输对齐的扇区或更大的块。例如，当前的 Nsight 文档报告了 L1 和 L2 中的 32 字节扇区，而 L2 请求可能覆盖 128 字节缓存行（Cache Line）中的最多四个扇区。 [3]

如果 warp 的线程需要相邻的 4 字节值，这些有用的字节可以共享少量扇区。如果相同的 32 个线程接触的地址分散得很远，则即使源代码每个线程仍然包含一条加载指令，硬件也可能需要许多扇区。

因此，空间局部性会影响该比率

η_{traffic}= 存储系统传输的程序字节请求的有用字节.

这不是一个通用的硬件指标，但它是一个有用的会计思想。低比率意味着移动花费在线程束（Warp）不使用的字节上。

空间局部性与时间局部性不同。内核（Kernel）可以一次传输完全连续的数据，从而提供出色的空间行为，但几乎没有时间重用。另一个内核可能会重复访问一小组分散的位置，从而在没有简单连续模式的情况下实现时间重用。

> 层次结构出于不同的原因奖励这两种局部性：空间局部性使每次传输更有用，而时间局部性使相同的传输数据在被驱逐之前再次有用。

合并是 GPU 特定的机制，它将 warp 的全局内存请求集转换为访问模式允许的尽可能少的内存事务。

16 工作组和复用距离

对应原稿第 20 页

工作集（Working Set）是需要在执行的某个阶段保持方便可用的数据子集。相关大小取决于问题：线程的实时标量、块的共享图块、SM 的活动数据或内核（Kernel）重复重用的全局区域都是不同规模的工作集。

当您将活动工作集（Working Set）与可用于保留它的存储进行比较时，缓存行为就变得更容易推理。

假设内核（Kernel）重复循环 8 MiB 数据，并且相关共享缓存有足够的有效空间用于该区域。重用可能会被很好地捕获。如果同一个内核循环数百兆字节，而许多其他客户端竞争缓存，则较大部分的请求将需要发送到更远的地方。

对于同一想法，重用距离是一个有用的直觉。在对同一值的两次访问之间，触及了多少不同的竞争数据？介入的足迹越大，小缓存保留该值的可能性就越小。确切的缓存行为还取决于映射、关联性、替换策略和并发流量，因此重用距离并不是一个完整的预测器。

这解释了为什么应该谨慎使用“适合 L2”。 40 MB 张量（Tensor）和 50 MB L2 并不能保证近乎完美的重用。其他数据、缓存行映射和并发客户端也很重要。同样，如果热子集很小或者访问模式重复重用某些区域，大于 L2 的工作集（Working Set）仍然可以实现许多命中。

> 容量比较提供了必要的线索，而不是缓存性能（Performance）证明。工作集大小、重用顺序、映射和竞争共同决定了实际捕获的重用量。

这也是为什么即使数学运算保持不变，性能（Performance）也会随着张量形状（Tensor Shape）而急剧变化：形状可以将活动工作集（Working Set）移动到缓存容量阈值。

17 缓存命中、未命中和下游流量

对应原稿第 21 页

缓存命中（Cache Hit）很有价值，因为它可以满足请求，而无需将相同的请求发送到更远的级别。因此，主要内存结果是流量过滤。

如果 1 TB 的请求到达 L2，并且大部分请求在那里得到服务，则 HBM 可能会看到远少于 1 TB 的读取流量。如果大多数请求未命中，HBM 会看到更多的需求。 Nsight 的内存表专门公开了命中率和扇区缺失，以使这种区别可见。[3]

命中率本身仍然不完整。考虑两个内核：

• 内核（Kernel）A 发出1000 万个请求，命中率为95%。

• 内核（Kernel）B 发出100 万个请求，命中率为80%。

内核（Kernel） A 具有更好的百分比，但从绝对值来看仍然可以向下游发送更多未命中。每个请求执行的有用工作量也很重要。

延迟（Latency）行为也可能有所不同。一个缓存级别的命中可能比传播更远的未命中更快，但确切的损失取决于体系结构和请求类型。一些专门的缓存路径（例如纹理获取）具有不应推广到普通全局加载的语义。[1][3]

> 评估缓存时，请查看下游保存的字节和请求，而不仅仅是命中率百分比。当高命中率从实际限制内核（Kernel）的路径中消除流量或延迟（Latency）时，它非常有用。

这种框架在数据量巨大的人工智能内核中变得尤为重要。重用方面的适度改进可以消除千兆字节的 HBM 流量，而微小侧面结构上惊人的命中率百分比可能对实际耗时（Wall-clock Time）几乎没有影响。

18 合并：组织线程束（Warp）的内存请求

对应原稿第 22 页

当 warp 中的线程发出全局内存指令时，GPU 将它们的地址组合到

满足线程束（Warp）所需的内存事务。良好的合并意味着这些事务包含线程实际使用的大部分字节。[1][2]

当前的 CUDA 示例对 32 字节事务中的全局内存服务进行建模。对于具体的 64 位情况，加载 32 个对齐的连续 8 字节值的完整线程束（Warp）需要 256 个有用字节。这些地址占用 8 个 32 字节段。[1][3]

车道编号不是规则。如果通道以不同的顺序访问相同的 32 个值，则所需段的集合可以保持不变。因此，合并与线程束（Warp）创建的地址占用有关，而不是强制 Lane i读取 element i.[1][2]

废物运动聚结不良。如果这 32 个有用的 8 字节值被分散，使得每个值都落在不同的 32 字节段中，则请求占用空间将增长到 32 个段，即使有用的有效载荷仍然是 256 字节。

对齐很重要，因为跨越附加段的连续请求可能需要额外的事务。缓存行为可以减轻某些错位的成本，但它并不能消除分散的访问模式。[2]

> 合并是一个流量效率的概念。它减少了 warp 请求所需的内存段数量。它不应该被简化为“线程必须始终访问连续的地址”。

AS07 将详细处理事务、对齐、跨步和特定于架构的合并规则。这里我们只需要了解为什么张量（Tensor）布局可以在不改变张量大小的情况下改变内存流量（Memory Traffic）。

19 工作示例：连续负载和跨步负载

对应原稿第 23 页

假设一个 warp 有 32 个活动线程，每个线程加载一个对齐的 8 字节值，并且访问足迹以 32 字节段计算。[1][3]

情况 A：跨步 1. Thread i读取 element i.

$32 \times 8 = 256$有用字节.

地址占用 8 个 32 字节段，因此 256 个段字节覆盖 256 个有用字节。

情况B：跨步 2. 线程 i读取元素$2i$.请求的值占用16 个段。 warp 仍然需要 256 个有用字节，而覆盖的段占用空间为 512 个字节。

情况 C：跨步 4. Thread i读取元素$4i$.每个请求的 8 字节值都从不同的 32 字节段开始，因此 32 段覆盖 1024 字节，共 256 个有用字节。

图案	段	段字节	有用分数
stride 1	8	256 B	100%
stride 2	16	512 B	50%
stride 4	32	1024 B	25%

步幅 1 步幅 2 步幅 4

这些计数描述了这个简化对齐示例中线程束（Warp）的段需求。如果请求在缓存中命中，则实际 HBM 流量可能会较低，并且特定于体系结构的路径可以包含其他详细信息。本课是关于请求几何的，而不是承诺每个段都成为 DRAM 事务。

> 在所有三种情况下，源代码可以包含相同数量的加载。内存系统不做同样多的工作。布局和步幅决定了传输的数据量是有用的。

20 平铺将容量转化为再利用

对应原稿第 24 页

平铺将一个大问题分成足够小的部分，以便在重用时保留在更近的存储级别中。

考虑一个玩具 GEMM，其中一个块负责 $C = AB$ 的 64×64 输出图块，并且完整的 K 维度为 32。对于 BF16 输入，该块需要 64×32 的 A 图块，并且 32×64 的 B 瓷砖

$$(64 \times 32 + 32 \times 64) \times 2 = 8192 \text{ bytes} = 8 \text{ KiB}.$$

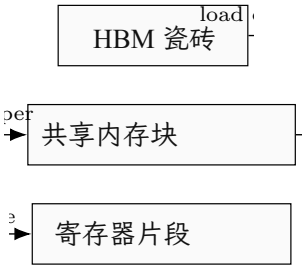
这 8 KiB 输入支持

$$2 \times 64 \times 64 \times 32 = 262{,}144$$

乘法和加法分别计算时的浮点（Floating-point）运算。

如果输出也是 BF16 且 $\beta = 0$，则写入最终的 64×64 结果会再增加 8 KiB。在这种简化的计算下，玩具问题对于 16 KiB 的输入/输出移动执行 262,144 次 FLOP，或者在该边界处每字节执行 16 次 FLOP。

重要的部分是重用。A 图块的每个元素都贡献许多输出列，B 的每个元素都贡献许多输出行。在较大的 GEMM 和 $K > 32$,中，块在多个 K 切片上重复此阶段，同时部分输出累加器仍接近计算。



每个图块重用时加载一次

> 平铺不会使张量（Tensor）变小。它更改了计划，以便在离开快速存储级别之前可以重用较小的工作集（Working Set）。

21 GEMM 也是一个内存重用问题

对应原稿第 25 页

矩阵乘法（Matrix Multiplication）以其算术吞吐量（Throughput）而闻名，但高性能（Performance）GEMM 也是精心设计的数据移动调度。

简单的循环可以从设备内存（Device Memory）中重复获取相同的矩阵（Matrix）元素。相反，阻塞实现将图块映射到线程块，在芯片上暂存可重用输入，将较小的片段加载到寄存器（Registers）中，并在写回结果之前累积许多乘积。

CUTLASS 准确地描述了这种层次结构：线程块提供粗略并行性，共享内存块保留用于协作线程束（Warps）的数据，寄存器（Register）片段保留重复矩阵（Matrix）指令的值。[14]

这就是为什么GEMM可以达到很高的算术强度（Arithmetic Intensity）。相同的字节被故意重复使用多次。然而，随着矩阵（Matrix）尺寸和图块形状的变化，这种重用可能会减弱。小矩阵、窄尺寸、笨拙的尾部或并行性不足可能无法像大型形状良好的 GEMM 那样有效地摊销运动。

内存层次结构（Memory Hierarchy）还造成资源权衡。较大的块可以增加重用，但会消耗更多的共享内存（Shared Memory）和寄存器（Registers）。更深层次的软件管道可以与计算重叠传输，但需要额外的缓冲。如果占用率（Occupancy）太低或造成存储体冲突，最大化再利用的设计仍然会失败。

> GEMM 的快速部分不仅仅是“Tensor Core 很快”。高效的 GEMM 安排数据移动（Data Movement），因此昂贵的矩阵（Matrix）硬件会从附近的存储中连续馈送，同时每个获取的图块都会重复使用多次。

这是通往后面系统主题的桥梁。 AS03 中的算术强度（Arithmetic Intensity）量化了每个字节的工作量。AS04 中的 Roofline 将这种强度与硬件天花板联系起来。共享内存和寄存器（Register）手册解释了使重用成为可能的片上分级细节。

移动 22 字节与执行算术运算

对应原稿第 26 页

内核（Kernel）的数学运算计数并不能告诉您它需要多少数据移动（Data Movement）。有用的第一个比率是

算术强度（Arithmetic Intensity） = 在选定的内存边界移动的 FLOPs 字节.

边界很重要。每个 HBM 字节、每个 L2 字节和每个 L1 字节的 FLOP 可能不同，因为缓存和片上重用过滤流量。 AS03会谨慎对待这一点。

考虑 FP32 向量（Vector）加法：

$$C_i = A_i + B_i.$$

忽略缓存重用并假设一次读取 A,一次读取 B,一次写入 C,，每个元素移动 12 个字节以进行一次浮点（Floating-point）加法：

1 FLOP 12 字节 ≈ 0.083 FLOP/字节.

几乎没有重用来放大每个字节的算术工作。

前几页中的平铺 GEMM 示例非常不同。在其简化的输入加输出计算下，一个小块已经达到每字节 16 FLOP，因为每个输入值在替换之前参与了许多乘加运算。

这并不能证明矢量加法总是受带宽限制，或者 GEMM 总是受计算限制。实际性能（Performance）取决于形状、启动开销（Launch Overhead）、缓存状态、精度（Precision）、硬件和其他瓶颈。该比率简单地揭示了有多少算术可用于分摊数据移动（Data Movement）。

> 如果第一个工作负载移动字节效率低下，而第二个工作负载重用数据良好，则较低的 FLOP 计数可能会比较高的 FLOP 计数运行得更慢。 FLOPs 和流量是独立的核算维度。

Roofline 模型形式化了这种关系，但 AS02 只需要直觉：AI 性能（Performance）取决于数学量和必须跨越受限接口的字节数。

23 为什么元素内核和 GEMM 的行为不同

对应原稿第 27 页

元素运算通常涉及一个大张量（Tensor），对每个元素执行少量算术，然后继续。矩阵乘法（Matrix Multiplication）重复组合相同的输入块以产生许多输出。这两种结构对内存层次结构（Memory Hierarchy）提出了截然不同的要求。

对于逐元素激活（Activation），可以读取一次值、转换一次并写入一次值。在该值离开内核（Kernel）之前，重用该值的机会有限。如果算术很便宜，那么读写张量（Tensor）的成本就会占主导地位。

对于 GEMM，阻止会导致故意重用。A的图块可以参与许多输出列，而B的图块可以参与许多输出行。因此，相同的加载字节可以提供更多的算术。

这种差异解释了为什么融合很重要。假设一个激活（Activation）从 HBM 读取一个大的 GEMM 输出，并写回另一个张量（Tensor）。如果激活可以作为 GEMM 尾声的一部分执行，而结果仍在附近，则大型中间体的一次往返可能会消失。 CUTLASS 明确将尾声视为可以将缩放（Scaling）和元素工作与矩阵（Matrix）结果相结合的地方。 [14]

融合不会自动带来好处。它会增加寄存器压力（Register Pressure），使调度复杂化，减少其他地方的重用，或阻止使用高度优化的库内核（Kernel）。 AS11 将处理这些权衡。

> 记忆问题不是“这个操作简单吗？”它是“每个级别必须跨越多少字节才能执行有用的工作？”当应用于数十亿个几乎没有重用的值时，单行元素公式可能会很昂贵。

24 注意力机制（Attention）也是一个数据移动问题

对应原稿第 28 页

稠密注意力（Dense Attention）结合了矩阵乘法（Matrix Multiplication）、softmax、掩蔽和另一种矩阵乘法。熟悉的等式

$$\varsigma\left(\frac{QK^T}{\sqrt{d}}\right.$$

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + B\right)V$$

没有指定中间分数和概率值必须位于何处。

简单的分解可以具体化 $N \times N$分数矩阵（Score Matrix），读取它以进行 softmax，写入概率，然后再次读取它们以进行 PV乘法。对于长序列，这些中间体足够大，以至于将它们移动到设备内存（Device Memory）中可能会成为主要成本。

内存层次结构（Memory Hierarchy）开启了另一种可能性：形成更小的分数块，在它们保留在芯片上时消耗它们，仅维护组合 softmax 块所需的状态，并避免将完整密集中间写入 HBM。

这是调度的改变，而不是对稠密注意力（Dense Attention）函数的改变。算术交互仍然密集。该实现只是选择一个内存调度，以更长时间地保留接近计算的有用中间状态。

> 方程指定数据依赖性，而不是存储决策。实现可以生成中间图块并立即使用它，而无需在设备内存（Device Memory）中分配完整的中间张量（Tensor）。

这个想法比注意力机制（Attention）更广泛。内核（Kernel）融合、重新计算、平铺和运算符调度都利用相同的原理：有时额外的本地工作比将大型中间移出到 HBM 并返回要便宜。

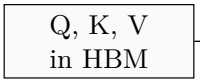
FlashAttention 是典型的 AI 示例，但确切的算法属于 Understanding AI 系列。这里我们用它只是为了揭示内存层次结构的教训。

25 FlashAttention 作为层次结构感知示例

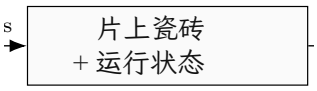
对应原稿第 29 页

密集 FlashAttention 避免将完整的分数和概率矩阵放入 HBM 驻留（Resident）中间体中。生成分数图块，在其仍在芯片上时将其合并到行归一化（Normalization）和输出状态中，然后将其丢弃。 The dense softmax-attention function being evaluated is unchanged.[13]

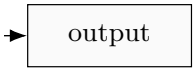
重要的记忆转换是：



在 HBM 中



加载图块更新



输出

其 IO 分析将 HBM 和有限片上存储之间的移动视为显式算法成本，并得出比本文内存模型（Model）下的标准物化计划更少的 HBM 访问。 [13]

这个例子纠正了两个常见的误解。

首先，减少 HBM 流量并不需要减少 FLOP。 FlashAttention 甚至可以在反向传播中重新计算一些中间产物，因为额外的算术可能比存储和重新读取大张量更便宜。

其次，“注意力机制（Attention）受记忆限制”过于宽泛。主要瓶颈取决于序列长度、头维度（Head Dimension）、精度（Precision）、硬件、前向与后向以及预填充（Prefill）与解码（Decode）。 A memory hierarchy tells us where costs can arise.它不允许我们为每一个注意力机制工作负载贴上一个瓶颈标签。

> 持久的教训不是“使用 FlashAttention”。就是算法设计可以包含内存层次结构（Memory Hierarchy）。中间体的创建、重用和废弃位置与定义它的方程式一样重要。

26 以不同的方式预填充（Prefill）和解码（Decode）压力记忆

对应原稿第 30 页

LLM 推理（Inference）至少包含两个不同的执行阶段。

Prefill 一起处理许多提示词元（Tokens）。矩阵（Matrix）维度更大，有很多查询（Query）行，优化的 GEMM/注意力机制（Attention）内核可以暴露大量的重用和并行性。 The computation often has more opportunity to amortize weight and activation movement across many operations.

解码（Decode）通常一次将每个序列推进一个新词元（Token）。在小批大小下，许多层操作变成矩阵（Matrix）-向量（Vector）或非常小的矩阵-矩阵形状。相同的模型（Model）权重仍然需要参与，但每个词元摊销这些读取的工作量要少得多。不断增长的键值缓存（KV Cache）也得靠attention来读取。

一个简单的下限思想实验说明了为什么内存带宽（Memory Bandwidth）很重要。考虑一个假设的密集 70 亿参数模型（Model），每个权重存储 2 个字节。其权重以十进制单位计算约为 14 GB。如果每个词元（Token）都需要一次全权重扫描，并且 H100 SXM 除了这些读取之外什么都可以维持其宣传的 3.35 TB/s，那么仅扫描一项就需要

$$14\ \mathrm{GB}/3.35\ \mathrm{TB/s} \approx 4.18\ \mathrm{ms},$$

在计算 KV 流量、激活、非权重读取、同步（Synchronization）或不完美的带宽使用之前，相当于大约 239 个词元（Token）/秒。[7]

这是一种用于检查数量级是否合理的上界估算，并非吞吐量（Throughput）预测。

> Prefill and decode use the same model but can occupy very different points in the compute-versus-memory trade space.在未测量实际形状和批次状态的情况下，切勿将“内存限制”或“计算限制”标签从一个阶段转移到另一阶段。

27 权重、激活和 KV 状态具有不同的生命周期

对应原稿第 31 页

人工智能系统移动几类主要的数据，它们的生命周期决定了内存层次结构（Memory Hierarchy）如何提供帮助。

权重在请求和词元（Tokens）之间是持久的。一个层在多个批次中重复使用相同的参数

元素、序列位置和未来的请求。然而，大型模型（Models）比片上缓存大得多，因此随着时间的推移，大部分权重流仍然来自设备内存（Device Memory）。

激活是在模型（Model）执行时产生的。 Their lifetime depends on training versus inference, fusion, checkpointing, and the operator graph.一些中间体会立即被消耗，而另一些则必须存活更长时间。

键值缓存（KV Cache）在自回归解码（Decode）过程中持续存在，因为未来的词元（Tokens）会处理早期位置的键和值。它的占用空间随着普通缓存注意力机制（Attention）中的序列长度而增长，因此长上下文解码可以产生大量的持久内存需求和读取流量。

这些类别的重用粒度也有所不同。权重图块可以在一个内核（Kernel）内的一批中的多个词元（Tokens）之间重复使用。激活（Activation）块只能在融合算子（Operator）内重复使用。在长会话中，每个解码（Decode）步骤可以重新访问 KV 块一次。

> 内存优化从生命周期开始。不仅要问“这个张量（Tensor）有多大？”而是“谁会重复使用它，多久，以及在什么范围内？”对于不可变权重的正确存储策略对于短期激活（Activation）图块来说可能是完全错误的。

这种分类还解释了为什么内存容量和带宽压力会随着工作负载而变化。培训商店更多状态。长上下文推理（Inference）增加了 KV 状态。小批量解码（Decode）可能会以有限的算术重用来重复流式传输权重。一个层次结构为所有这些服务提供服务，但流量模式不同。

28 训练（Training）和推理（Inference）移动不同的字节

对应原稿第 32 页

训练（Training）和推理（Inference）执行相关的模型（Model）方程，但它们的记忆行为并不相同。

在训练（Training）过程中，前向传播（Forward Pass）会产生反向传播所需的激活。向后计算梯度并可以读取保存的激活或重新计算它们。优化器维护附加参数关联状态。因此，该系统移动和存储的量远远多于模型（Model）重量。

激活（Activation）检查点通过存储更少的激活并在向后过程中重新计算选定的前向工作来改变这种平衡。这是用额外的算术换取较低的激活能力，这是用计算换取内存的一个明确例子。 AS23 将深入介绍该技术。

推理（Inference）消除了梯度（Gradient）和优化器状态流量，但它引入了自己的机制。 Prefill 一次处理多个词元（Tokens）。自回归解码（Decode）跨步骤保留 KV 状态，并且在小批大小下，每个生成的词元对大权重矩阵的重用可能有限。

因此，相同的优化可能具有不同的价值。保存激活（Activation）写入在训练（Training）中可能非常重要。在长上下文解码（Decode）中，减少持久 KV 读取可能更重要。增加批大小（Batch Size）可以改善推理（Inference）中的权重重用，但可能会增加在线服务中的排队延迟（Latency）。

> 不要使用一个称为“模型（Model）内存”的数字来描述训练（Training）和推理（Inference）。权重、梯度、优化器状态、激活、临时工作区和 键值缓存（KV Cache）具有不同的生命周期和流量模式。

内存层次结构（Memory Hierarchy）是这些差异背后的硬件基础。分片、检查点、卸载（Offloading）、连续批处理（Continuous Batching）和 KV 分页等系统级技术决定存在哪些字节以及何时存在。内核决定每个活动工作集（Working Set）在 GPU 中移动的效率。

29 容量和带宽是独立的硬件属性

对应原稿第 33 页

当前的加速器使这种区别具体化。下表使用供应商公布的每 GPU 容量和峰值内存带宽（Memory Bandwidth），时间范围为引用的产品文档。这不是绩效排名。

GPU	内存类型	容量	峰值带宽
NVIDIA H100 SXM	HBM3	80 GB	3.35 TB/s[7]
NVIDIA H200 SXM	HBM3e	141 GB	4.8 TB/s[8]
NVIDIA B200 SXM	HBM3e	180 GB	up to 8 TB/s[9]
AMD MI300X	HBM3	192 GB	5.3 TB/s[10]
AMD MI355X	HBM3E	288 GB	8 TB/s[11]

NVIDIA H100 SXM NVIDIA H200 SXM NVIDIA B200 SXM AMD MI300X AMD MI355X

HBM3 HBM3e HBM3e HBM3 HBM3E

规格本身说明了为什么两个色谱柱必须分开。从H100 SXM到H200 SXM，公布的内存容量标签从80 GB上升到141 GB，约增加了76%，而峰值内存带宽（Memory Bandwidth）从3.35 TB / s上升到4.8 TB / s，约43%。容量和带宽之间没有固定的换算。它们是独立的硬件属性，对工作负载具有不同的影响。[7][8]

> 容量回答“工作集（Working Set）可以驻留在设备上吗？”带宽回答“流量能够多快维持？”一个好的系统需要两者都足以满足其工作负载。

30 理论带宽与实际带宽

对应原稿第 34 页

供应商带宽规格描述了峰值硬件功能。内核（Kernel）所达到的速率必须根据一段时间内实际穿过接口的字节来测量。

CUDA 最佳实践定义了以下形式的有效带宽计算：[2]

$$B_{\text{eff}} = \frac{B_{\text{read}} + B_{\text{write}}}{t},$$

分子必须使用明确的字节计数约定，并且单位必须与用于比较的峰值数相匹配。

假设一个内核（Kernel）在 5 毫秒内读取 4 GiB 并写入 4 GiB。这是 8 GiB 的有用应用程序流量。以十进制字节每秒为单位，

$$\frac{8 \times 2^{30}}{0.005} \approx 1.72 \times 10^{12}\ \text{B/s} = 1.72\ \text{TB/s}.$$

这仍然没有告诉我们确切的 DRAM 流量。缓存、写入策略、ECC 行为、压缩和额外事务可能会使物理移动与有用字节计数不同。

因此，Nsight 报告 L1、L2 和设备内存（Device Memory）的字节和吞吐量（Throughput）的硬件计数器，而不是要求开发人员从张量（Tensor）大小推断所有内容。 [3]

> 三个数字都可以不同：供应商峰值带宽、根据算法计算出的有用有效带宽以及特定内存接口处硬件观察到的字节/秒。说明你指的是哪一个。

这也是为什么跨纸带宽数字需要谨慎。不同的精度、张量（Tensor）形状、缓存状态、时钟、ECC 设置和字节计数约定可能会使表面上相似的 GB/s 数字变得无法比较。

31 缓存友好并不自动意味着速度快

对应原稿第 35 页

仅当内存限制足以使改进发挥作用并且该技术不会在其他地方引入更大的成本时，改进局部性才有价值。

共享内存（Shared Memory）平铺策略可能会减少 HBM 流量，但会消耗足够的共享内存来减少块驻留。缓存持久布局可以提高 L2 命中率，但会增加索引或同步（Synchronization）开销。融合内核（Fused Kernel）可能会消除中间写入，但会产生寄存器压力（Register Pressure），从而降低有用的并行性。

即使在内存系统内，一个优化级别也可能揭示另一个瓶颈。减少 HBM 流量可能会将限制转移到 L2 带宽。提高 L2 重用可能会暴露指令问题或 Tensor Core 利用率。 Nsight 的内存分析明确处理了几个不同的限制：繁忙的内存单元、单元之间的带宽以及内存指令管道的吞吐量（Throughput）。 [3]

因此，正确的优化循环是因果关系：

1. 识别实际限制进度的接口或依赖项，

2. 改变时间表或布局以降低成本，

3. 重新测量整个内核（Kernel），

4.检查接下来限制了什么。

> 更好的缓存命中（Cache Hit）率是一个中间指标。目标是缩短实际耗时（Wall-clock Time）、提高吞吐量（Throughput）、降低尾部延迟（Latency）或其他明确的应用程序指标。如果应用程序指标没有改善，则缓存优化可能无关紧要或被其他成本所抵消。

这是一门重复的系统课程。占用率（Occupancy）、利用率、缓存命中（Cache Hit）率和带宽是诊断信号。它们本身都不是最终目标。

32 高带宽并不意味着低延迟（Latency）

对应原稿第 36 页

带宽和延迟（Latency）描述了内存路径的不同属性。

内存接口可以同时支持许多传输，因此可以提供巨大的聚合带宽，而单个相关负载仍然需要大量时间才能完成。这些是路径的不同属性。

真实的 GPU 内存系统包含队列、库、缓存、聚结器和许多未完成的请求。观察到的依赖项延迟（Latency）会随着命中位置、争用和请求并发性而变化，因此一个永恒的“HBM 延迟”数字通常会产生误导。

具有许多独立请求的内核（Kernel）可以填充内存管道并接近高带宽。指针追逐内核一次可能只公开一个相关请求，从而阻止系统构建相同级别的并发性。两者都可以访问相同的HBM。

这也解释了为什么延迟（Latency）隐藏和延迟减少是不同的。缓存命中（Cache Hit）可以减少请求传输的距离。 Warp 调度可能会将长请求与不相关的工作重叠。底层请求不一定会变得更快。在实际运行执行中可以看到更少的等待时间。

> 带宽询问聚合流量。延迟（Latency）询问依赖性。 AI 内核通常需要足够的请求级和线程束（Warp）级并行性，才能将高带宽内存（High-bandwidth Memory）系统转变为高应用程序吞吐量（Throughput）。

AS03和AS04将添加带宽上限的定量模型（Models）。 AS09 将返回驻留和独立线程束（Warps）如何帮助覆盖内存延迟（Latency）。

33 NVIDIA 和 AMD：相同的问题，不同的词汇

对应原稿第 37 页

物理目标是跨供应商共享的：保留靠近计算的有用数据并避免不必要的片外移动。名称和确切的组织有所不同。

概念	NVIDIA CUDA 示例	AMD cDNA 示例
Thread-private registers	register file	VGPR / SGPR files
Explicit cooperative storage	shared memory	LDS
First data cache near compute	L1/TEX organization	per-CU L1 vector cache
Shared deeper cache	L2	L2 plus architecture-specific last-level cache hierarchy
大设备内存执行组	HBM on H100/H200/B200 warp	HBM on MI300/MI350 series wavefront

线程私有寄存器（Registers） 显式协作存储 靠近计算的第一个数据缓存 共享更深的缓存

寄存器文件（Register File） 共享内存（Shared Memory） L1/TEX 组织

H100/H200/B200 经纱上的 HBM

VGPR / SGPR 文件 LDS 每个 CU L1 矢量缓存 L2 以及 MI300/MI350 系列波前（Wavefront）上特定于架构的最后一级缓存层次结构 HBM

AMD 的 CDNA 3 ISA 文档描述了每个计算单元 64 KiB 的 LDS 以及通过更深的共享缓存提供每 CU L1 缓存的内存层次结构（Memory Hierarchy）。当前的 ROCm 架构文档列出了 MI300X 的每个 CU 32 KiB L1 矢量缓存、32 MiB 聚合 L2 以及该产品系列的 256 MiB 末级缓存结构。[12][16]

该表是一种翻译辅助工具，而不是一对一微架构等效性的声明。 NVIDIA 共享内存（Shared Memory）和 AMD LDS 具有类似的程序员管理的协作角色，但它们的容量、存储体结构、指令和周围的缓存层次结构是特定于供应商的。

> 先讲原理，再讲厂商机制。“在芯片上部署可重复使用的图块”是可移植的推理。“使用这个精确的共享内存剥离”或“假设这个缓存大小”是特定于体系结构的建议。

即使 GPU 各代发生变化，这种分离也使手册保持有用。

34 代架构改变了细节

对应原稿第 38 页

内存层次结构概念老化缓慢，而容量和机制老化很快。当前的一些示例说明了为什么实施细节必须具有时间范围。

料斗 H100。 NVIDIA 记录了每个 SM 的 50 MB L2 缓存和 256 KB 组合 L1/纹理/共享内存（Shared Memory）容量，其中最多 228 KB 配置为共享内存。 H100 SXM 产品规格列出了 80 GB HBM3 和 3.35 TB/s 内存带宽（Memory Bandwidth）。[5][7]

布莱克威尔. NVIDIA 当前的调优指南为 B200 级计算能力 10.0 保留了 256 KB 组合的 L1/纹理/共享资源，并为 GB200 保留了 126 MB 的 L2 文档。 HGX B200 规格列出了 180 GB HBM3e，每个 GPU 高达 8 TB/s。[6][9]

AMD CDNA。 MI300X 产品文档列出了 192 GB HBM3，速度为 5.3 TB/s。 MI355X 基于 CDNA 4，列出 288 GB HBM3E，速度为 8 TB/s。 ROCm 文档还显示了 LDS 容量的代际变化，MI350 系列部件比 MI300 系列部件每个 CU 暴露更多的 LDS。[10][11][16]

这些数字都不会改变基本的层次结构课程。较大的 L2 可以保留较大的工作集（Working Set）。更多的 HBM 带宽提高了片外上限。更多共享内存（Shared Memory）可以启用更大的图块或更深的管道。但软件仍然需要可以利用这些资源的访问模式。

> 出版规则。在未来版本中重用此页面之前，根据当前架构文档重新检查容量、支持的分拆、缓存控制和产品带宽。不要仅仅因为此快照已过时就给常青章节添加时间戳。

35 种常见误解

对应原稿第 39 页

“如果模型（Model）适合 HBM，那么记忆问题就解决了。”

不。容缩放因子示字节可以驻留在此处。运行时间仍然会受到流量、带宽或延迟（Latency）的限制。

“全局内存（Global Memory）意味着 HBM。”

不。全局内存（Global Memory）是一个地址空间。全局负载在到达设备存储器之前可以由 L1 或 L2 提供服务。

“本地内存靠近线程。”

不。它的范围是本地的，但 CUDA 通过设备内存（Device Memory）支持它。[1]

“共享内存（Shared Memory）只是软件控制的 L1。”

太松了。它们可能在最新的 NVIDIA GPU 上共享物理容量，但共享内存（Shared Memory）具有显式寻址、范围、同步（Synchronization）和存储体行为。

“缓存命中（Cache Hit）率意味着内核（Kernel）高效。”

不一定。内核（Kernel）可能在其他地方受到限制，或者缓存可能只推理服务（Serving）于总工作的一小部分。

“HBM 具有高带宽，因此单个加载速度很快。”

带宽是总流量。 Latency is a dependency delay.高带宽并不能消除延迟（Latency）。

“合并意味着线程 i必须读取元素 i.”

不。我们的目标是用尽可能少的所需内存段覆盖线程束（Warp）的地址。排列的访问仍然可以合并。[1][2]

“更多的重复利用总是更好。”

重用机制会消耗寄存器（Registers）、共享内存（Shared Memory）、同步（Synchronization）和占用。节省的流量必须证明这些成本是合理的。

> 大多数记忆神话都来自于将多个级别或指标扁平化为一个单词。将地址空间、物理位置、容量、流量、带宽、延迟（Latency）和重用分开。

36 一套实用的内存理解框架（Mental Model）

对应原稿第 40 页

当 AI 内核（Kernel）意外缓慢时，请按此顺序遍历层次结构。

1. 识别活动数据。权重、激活、KV 状态、临时图块、输出和元数据具有不同的大小和生命周期。

2. 选择边界。您是否在考虑注册/共享流量、L1/L2 流量、HBM 流量或主机设备流量？如果没有边界，“移动的字节数”就毫无意义。

3. 估计有用字节。在明确的假设下计算算法隐含的最小读取和写入次数。

4.寻找重用。哪些值在被逐出之前可以在寄存器（Registers）、共享内存（Shared Memory）或缓存中重用？

5. 检查通道几何形状。 Are warp requests coalesced?步幅或布局是否会导致额外的扇区？

6. 将容量与速率分开。 Does the working set fit?如果是，所需的流量是否仍会超出可用带宽或延迟（Latency）预算？

7. 分析实际路径。 Nsight 的内存工作负载分析公开了 L1、L2 和 DRAM 字节、命中率、请求和吞吐量（Throughput）。使用计数器来测试模型（Model），而不是将其视为证据。[3]

8. 重新测量端到端。仅当应用程序指标改善时，内存优化才会成功。

> 紧凑型。数据从生命周期和重用模式开始。内核（Kernel）将数据映射到寄存器（Registers）、显式片上存储、硬件缓存和设备内存（Device Memory）。性能（Performance）取决于每次传输在字节必须再次移动之前实现了多少有用的工作。

该模型（Model）是下一本手册的基础。 AS03将量化算术强度（Arithmetic Intensity）。 AS04 将根据计算和带宽上限来安排工作负载。 AS07-AS09 将重点讨论合并、共享内存（Shared Memory）、寄存器（Registers）和占用。 AS11 将解释为什么融合可以删除整个层次结构的行程。

参考文献和进一步阅读

对应原稿第 41 页

[1] NVIDIA. CUDA Programming Guide, current documentation. CUDA 编程指南，最新文档。 Memory spaces, global/local/shared memory, devicememory accesses, cache control, and memory-throughput guidance. https://docs.nvidia.com/cuda/cuda-programming-guide/

原题名与书目信息：[1] NVIDIA. CUDA Programming Guide, current documentation. Memory spaces, global/local/shared memory, devicememory accesses, cache control, and memory-throughput guidance. https://docs.nvidia.com/cuda/cuda-programming-guide/

[2] NVIDIA. CUDA C++ Best Practices Guide, current documentation. CUDA C++ 最佳实践指南，当前文档。 Coalescing, bandwidth calculations, sharedmemory reuse, and memory optimization. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

原题名与书目信息：[2] NVIDIA. CUDA C++ Best Practices Guide, current documentation. Coalescing, bandwidth calculations, sharedmemory reuse, and memory optimization. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[3] NVIDIA. Nsight Compute Profiling Guide. Nsight 计算分析指南。内存工作负载分析、L1/TEX、L2、DRAM、扇区、请求、命中率和吞吐量（Throughput）指标。 https://docs.nvidia.com/nsight-compute/ProfilingGuide/

原题名与书目信息：[3] NVIDIA. Nsight Compute Profiling Guide. Memory Workload Analysis, L1/TEX, L2, DRAM, sectors, requests, hit rates, and throughput metrics. https://docs.nvidia.com/nsight-compute/ProfilingGuide/

[4] NVIDIA. CUDA Programming Guide: Compute Capabilities. CUDA 编程指南：计算功能。 Register, shared-memory, and cache resource limits by compute capability. https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html

原题名与书目信息：[4] NVIDIA. CUDA Programming Guide: Compute Capabilities. Register, shared-memory, and cache resource limits by compute capability. https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html

[5] NVIDIA。 13.4. 料斗调整指南H100 内存系统、L2、共享内存/L1 资源和 HBM 详细信息。 https://docs.nvidia.com/cuda/hopper-tuning-guide/

原题名与书目信息：[5] NVIDIA. Hopper Tuning Guide 13.4. H100 memory system, L2, shared-memory/L1 resource, and HBM details. https://docs.nvidia.com/cuda/hopper-tuning-guide/

[6] NVIDIA。布莱克威尔调整指南 13.4。 B200/GB200 内存系统详细信息和共享内存/L1 资源行为。 https://docs.nvidia.com/cuda/blackwell-tuning-guide/

原题名与书目信息：[6] NVIDIA. Blackwell Tuning Guide 13.4. B200/GB200 memory-system details and shared-memory/L1 resource behavior. https://docs.nvidia.com/cuda/blackwell-tuning-guide/

[7] NVIDIA。 H100 Tensor Core GPU 产品规格。 H100 SXM 为 80 GB HBM3 和 3.35 TB/s。 https://www.nvidia.com/en-us/data-center/h100/

原题名与书目信息：[7] NVIDIA. H100 Tensor Core GPU product specifications. 80 GB HBM3 and 3.35 TB/s for H100 SXM. https://www.nvidia.com/en-us/data-center/h100/

[8] NVIDIA。 H200 Tensor Core GPU 产品规格。 H200 SXM 为 141 GB HBM3e 和 4.8 TB/s。 https://www.nvidia.com/en-us/data-center/h200/

原题名与书目信息：[8] NVIDIA. H200 Tensor Core GPU product specifications. 141 GB HBM3e and 4.8 TB/s for H200 SXM. https://www.nvidia.com/en-us/data-center/h200/

[9] NVIDIA. HGX AI Factory Reference Architecture: Components. HGX AI 工厂参考架构：组件。 H100/H200/B200内存容量和带宽表。 https://docs.nvidia.com/enterprise-reference-architectures/hgx-ai-factory-h100-h200-b200/latest/components. html

原题名与书目信息：[9] NVIDIA. HGX AI Factory Reference Architecture: Components. H100/H200/B200 memory capacity and bandwidth table. https://docs.nvidia.com/enterprise-reference-architectures/hgx-ai-factory-h100-h200-b200/latest/components. html

[10] AMD。 AMD Instinct MI300X 加速器。 192 GB HBM3，5.3 TB/s，以及当前产品规格。 https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html

原题名与书目信息：[10] AMD. AMD Instinct MI300X Accelerators. 192 GB HBM3, 5.3 TB/s, and current product specifications. https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html

[11] AMD。 AMD 本能 MI355X GPU。 288 GB HBM3E、8 TB/s 和 CDNA 4 产品规格。 https://www.amd.com/en/products/accelerators/instinct/mi350/mi355x.html

原题名与书目信息：[11] AMD. AMD Instinct MI355X GPUs. 288 GB HBM3E, 8 TB/s, and CDNA 4 product specifications. https://www.amd.com/en/products/accelerators/instinct/mi350/mi355x.html

[12] AMD. AMD Instinct MI300 CDNA 3 Instruction Set Architecture / microarchitecture documentation. AMD Instinct MI300 CDNA 3 指令集架构/微架构文档。 LDS、L1、L2 和设备内存层次结构（Memory Hierarchy）。 https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html

原题名与书目信息：[12] AMD. AMD Instinct MI300 CDNA 3 Instruction Set Architecture / microarchitecture documentation. LDS, L1, L2, and device-memory hierarchy. https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html

[13] Dao, T. et al. FlashAttention：具有 IO 感知能力的快速、内存高效的精确注意力（Exact Attention）。 NeurIPS 2022.https://arxiv.org/abs/2205.14135

原题名与书目信息：[13] Dao, T. et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. https://arxiv.org/abs/2205.14135

[14] NVIDIA CUTLASS。 CUDA 中的高效 GEMM。分层平铺、共享内存和寄存器（Register）重用以及流水线。 https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html

原题名与书目信息：[14] NVIDIA CUTLASS. Efficient GEMM in CUDA. Hierarchical tiling, shared-memory and register reuse, and pipelining. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html

[15] Williams, S., Waterman, A., Patterson, D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. ACM 通讯，2009 年。

原题名与书目信息：[15] Williams, S., Waterman, A., Patterson, D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. Communications of the ACM, 2009.

https://escholarship.org/uc/item/5tz795vq

[16] AMD ROCm. AMD GPU 架构文档，包括 CDNA 3/CDNA 4 架构参考和 MI300/MI350 硬件描述。https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html

原题名与书目信息：[16] AMD ROCm. AMD GPU architecture documentation, including CDNA 3/CDNA 4 architecture references and MI300/MI350 hardware descriptions. https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html

参考图：针对哪个问题阅读什么内容

对应原稿第 42 页

问题 最佳起始来源

CUDA的内存空间及其范围是多少？ CUDA 编程指南 [1] 本地内存和寄存器（Register）溢出如何表现？ CUDA 编程指南 [1]、Nsight 计算 [3] CUDA 示例中当前使用的合并模型（Model）是什么？

CUDA 编程指南 [1]，最佳实践 [2]

如何区分理论带宽和有效带宽？

CUDA 最佳实践 [2]

如何检查 L1、L2 和 DRAM 流量？ Nsight 计算分析指南 [3] 当前每个 SM 寄存器（Register）/共享内存限制是什么？

计算能力 [4]

H100 的内存系统有何特点？ Hopper 调校指南 [5]、H100 规格 [7] B200/GB200 的内存系统有何特点？ Blackwell 调优指南 [6]、HGX B200 规格 [9] 当前的 AMD 内存层次结构示例是什么？ AMD 架构文档 [12,16]，MI300X/MI355X 规格

[10,11]

为什么GEMM通过共享内存（Shared Memory）和寄存器（Registers）暂存数据？

CUTLASS Efficient GEMM [14]

Why is FlashAttention a memory-hierarchy example? FlashAttention 论文 [13] 算术强度（Arithmetic Intensity）接下来会走向何方？ 屋顶线纸 [15]

面向人工智能工程师、开发人员和认真学习者的简短技术手册